Java Collections Framework

List, Set, Map, TreeSet, Comparable and Comparator -
with class diagrams and live demos

Agenda

- Why collections? Arrays vs collections
- Collection framework and the API class diagram
- Generics: type-safe collections
- List: ArrayList and LinkedList (CRUD and searching)
- Set: HashSet, LinkedHashSet, TreeSet
- Map: HashMap, TreeMap and friends
- Sorting objects: Comparable vs Comparator
- Collections utility class, choosing the right collection, assignment

Why Collections? Arrays vs Collections

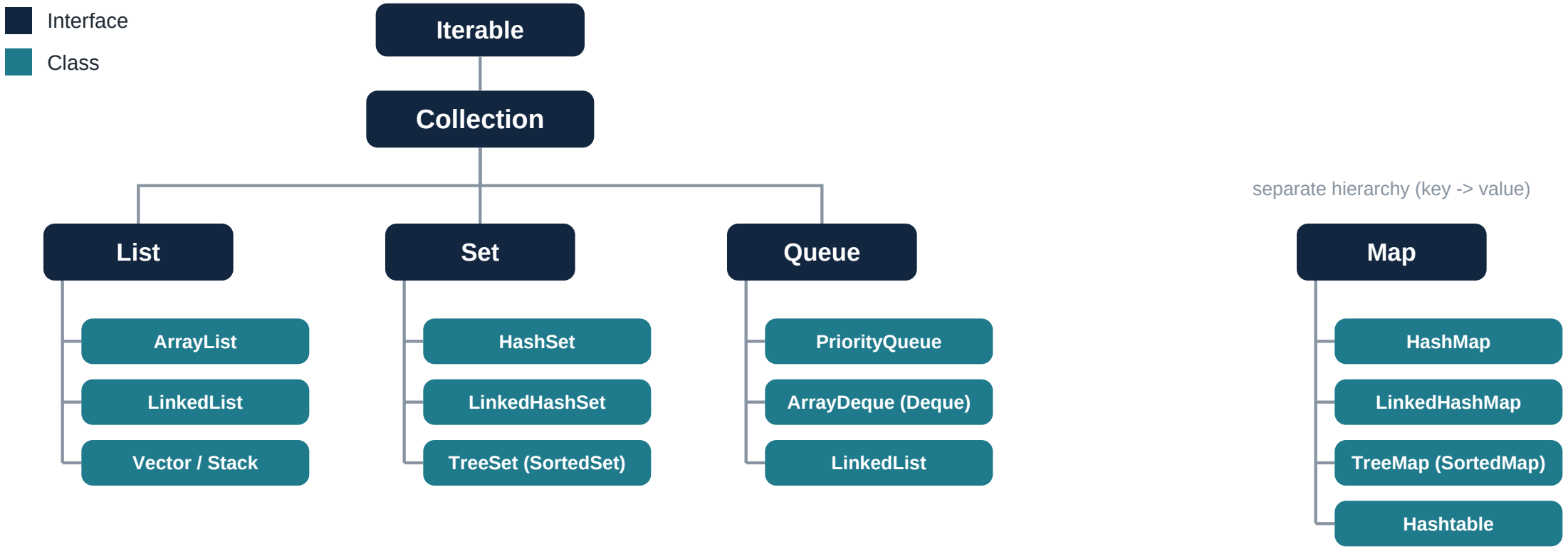
Feature	Array	Collection
Size	fixed once created	grows and shrinks automatically
Data	primitives or objects	objects only (wrappers for primitives)
Operations	only index access, length	add, remove, contains, sort, search ...
Structures	one: indexed sequence	list, set, map, queue, tree ...
Type safety	checked by compiler	checked using generics

- A collection is an object that groups many objects together and gives ready-made methods to manage them

Collection Framework - Big Picture

- A set of ready-made interfaces, classes and algorithms in the java.util package
- Interfaces define WHAT you can do: List, Set, Queue, Map
- Implementations define HOW it is stored: ArrayList, HashSet, TreeMap ...
- Algorithms: the Collections class (sort, reverse, max ...)
- Three names beginners confuse:
  - Collection = the root interface of List, Set and Queue
  - Collections = a utility class with static helper methods
  - Map = key-value store; NOT a child of Collection
- Program to the interface: List<String> list = new ArrayList<>();

Collection API - Class Diagram



Collections = utility class (sort, reverse, max, min ...) Comparable / Comparator = sorting rules

Generics - Type-Safe Collections

```
// WITHOUT generics (raw type): anything goes
List list = new ArrayList();
list.add("Amit");
list.add(100);                              // compiles!
String s = (String) list.get(1);            // ClassCastException at RUN time

// WITH generics: the type is checked at COMPILE time
List<String> students = new ArrayList<>();
students.add("Amit");
// students.add(100);                       // compile error: caught early
String name = students.get(0);              // no cast needed

Map<Integer, String> map = new HashMap<>(); // key type, value type
```

Key points

- <String> tells the compiler what the collection may hold
- Errors move from run time to compile time
- <> (diamond) lets Java infer the type on the right
- Our Employee demos use raw Comparable/Comparator, so they need casts

List - Ordered, Allows Duplicates



students.get(1) -> Priya **duplicates are allowed (Priya twice)**

- Keeps elements in insertion order; every element has an index starting at 0
- Duplicates allowed; null allowed
- Main methods: add, get, set, remove, contains, indexOf, lastIndexOf, size
- Implementations: ArrayList (array inside), LinkedList (nodes), Vector / Stack (legacy)

ArrayList vs LinkedList - Inside

ArrayList: one resizable block of memory



LinkedList: separate nodes joined by references



- ArrayList: fast get(i) because the address is calculated; insert in the middle shifts elements
- LinkedList: fast insert/delete (just change links); get(i) must walk node by node
- Rule of thumb: ArrayList by default; LinkedList when you add/remove often in the middle

ArrayList vs LinkedList - Comparison

Operation	ArrayList	LinkedList
get(index)	fast	slow (walks the nodes)
add at the end	fast	fast
insert / remove in the middle	slow (shifts elements)	fast (changes links)
Memory	less	more (pointers per node)
Best when	mostly reading	frequent insert / delete

Demo: ListCRUDDemo - Create, Read, Update, Delete

```
List<String> students = new ArrayList<>();

// CREATE
students.add("Amit");  students.add("Priya");  students.add("Rahul");
students.add("Priya");                         // duplicate is allowed
System.out.println("After Create: " + students);

// READ
for (String student : students) { System.out.println(student); }
System.out.println("Student at index 1: " + students.get(1));

// UPDATE
students.set(2, "Sneha");                      // replace at index 2
System.out.println("After Update: " + students);

// DELETE
students.remove(0);                            // remove at index 0
System.out.println("After Delete: " + students);
```

Key points

- add / get / set / remove = Create / Read / Update / Delete
- Index-based update: set(index, value)
- remove(0) shifts the rest left

Output

```
After Create: [Amit, Priya, Rahul, Priya]
Amit
Priya
Rahul
Priya
Student at index 1: Priya
After Update: [Amit, Priya, Sneha, Priya]
After Delete: [Priya, Sneha, Priya]
```

Demo: ListCRUDDemo - Searching

```
// students is now [Priya, Sneha, Priya]
System.out.println("Is Rahul present? " + students.contains("Rahul"));

String searchName = "Priya";
int index = students.indexOf(searchName);          // FIRST occurrence
if (index != -1) { System.out.println("First position: " + index); }
else             { System.out.println("Student not found"); }

System.out.println("Last position: " + students.lastIndexOf("Priya"));

// ALL occurrences: loop with an index
System.out.println("All matching positions:");
for (int i = 0; i < students.size(); i++) {
    if (students.get(i).equals(searchName)) {
        System.out.println("Found at index: " + i);
    }
}
```

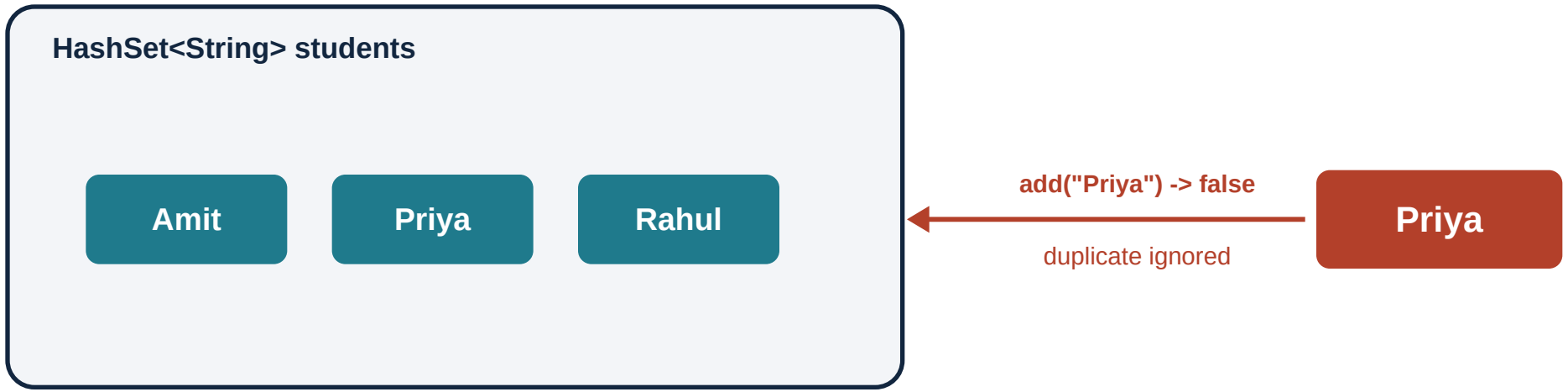
Key points

- indexOf returns -1 when not found
- lastIndexOf gives the last match
- Loop + equals finds every match

Output

```
Is Rahul present? false
First position: 0
Last position: 2
All matching positions:
Found at index: 0
Found at index: 2
```

Set - No Duplicates



- A Set stores only unique elements; adding a duplicate is silently ignored
- No index: you cannot get(i) or set(i, value)
- To 'update': remove the old value, add the new one
- Implementations: HashSet (no order), LinkedHashSet (insertion order), TreeSet (sorted)

HashSet vs LinkedHashSet vs TreeSet

Class	Order	Duplicates	Null	Internal structure
HashSet	not guaranteed	No	one null	hash table
LinkedHashSet	insertion order	No	one null	hash table + linked list
TreeSet	sorted (natural/custom)	No	not allowed	red-black tree

- Speed: HashSet is fastest; TreeSet is slower but keeps elements sorted
- TreeSet uses compareTo / compare to find the position AND to detect duplicates

Demo: SetCRUDDemo - HashSet

```
Set<String> students = new HashSet<>();

// CREATE
students.add("Amit");  students.add("Priya");  students.add("Rahul");
students.add("Priya");                        // duplicate is ignored
System.out.println("After Create: " + students);

// READ
for (String student : students) { System.out.println(student); }
System.out.println("Is Priya present? " + students.contains("Priya"));

// UPDATE: no index, so remove old + add new
students.remove("Rahul");
students.add("Sneha");

// DELETE
students.remove("Amit");
System.out.println("Total students: " + students.size());
```

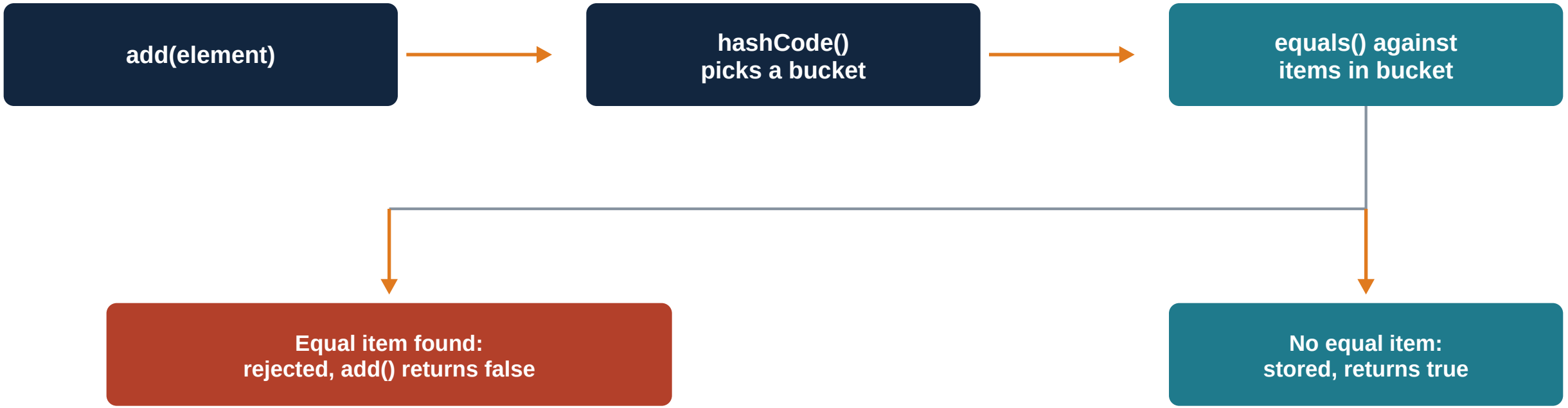
Key points

- Priya was added twice but stored once
- HashSet order is NOT insertion order
- Order may differ on another JDK or machine

Output

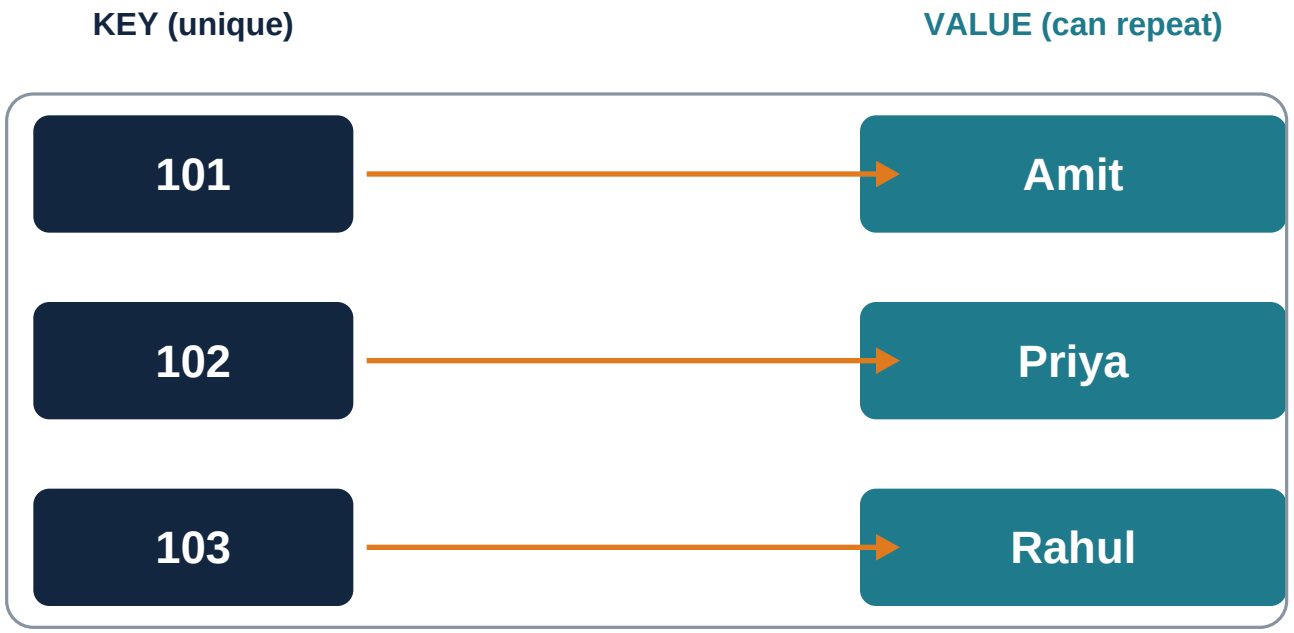
```
After Create: [Rahul, Priya, Amit]
Rahul
Priya
Amit
Is Priya present? true
After Update: [Priya, Sneha, Amit]
After Delete: [Priya, Sneha]
Total students: 2
```

How a HashSet Detects Duplicates



- Same hashCode first, then equals() decides - String and Integer already implement both
- For your own classes in a HashSet, override hashCode() and equals()
- TreeSet does NOT use hashCode: it uses compareTo() / compare() (result 0 = duplicate)

Map - Key-Value Pairs



each key->value pair is an Entry: Map.Entry<Integer,String>

Key points

- Stores data as key -> value pairs, like a dictionary
- Keys are unique; putting an existing key REPLACES its value
- Values may repeat; get(key) is the fast way to look up
- Not part of Collection; has its own hierarchy

Demo: MapCRUDDemo - HashMap

```
Map<Integer, String> students = new HashMap<>();

// CREATE
students.put(101, "Amit");  students.put(102, "Priya");  students.put(103, "Rahul");

// READ
System.out.println("Student with ID 102: " + students.get(102));
for (Map.Entry<Integer, String> entry : students.entrySet()) {
    System.out.println("ID: " + entry.getKey() + ", Name: " + entry.getValue());
}

// UPDATE: same key => value replaced
students.put(102, "Sneha");
// CREATE: new key => added
students.put(104, "Neha");

// DELETE by key
students.remove(101);

students.containsKey(103);          // true
students.containsValue("Priya");    // false
```

Key points

- put() both creates and updates
- remove(key) deletes the pair
- containsKey is fast; containsValue scans

Output

```
After Create: {101=Amit, 102=Priya, 103=Rahul}
Student with ID 102: Priya
ID: 101, Name: Amit
ID: 102, Name: Priya
ID: 103, Name: Rahul
After Update: {101=Amit, 102=Sneha, 103=Rahul}
After adding ID 104: {101=Amit, 102=Sneha, 103=Rahul,
    104=Neha}
After Delete: {102=Sneha, 103=Rahul, 104=Neha}
Does ID 103 exist? true
Is Priya present? false
```

HashMap vs LinkedHashMap vs TreeMap vs Hashtable

Class	Order of keys	Null	Notes
HashMap	not guaranteed	1 null key, many null values	fastest, most used
LinkedHashMap	insertion order	1 null key	remembers order
TreeMap	sorted by key	no null key	uses compareTo / compare
Hashtable	not guaranteed	no nulls at all	legacy, synchronized

Iterating Collections

```
// List and Set: enhanced for loop
for (String s : students) { System.out.println(s); }

// Iterator: lets you remove safely while looping
Iterator<String> it = students.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("Priya")) { it.remove(); }
}

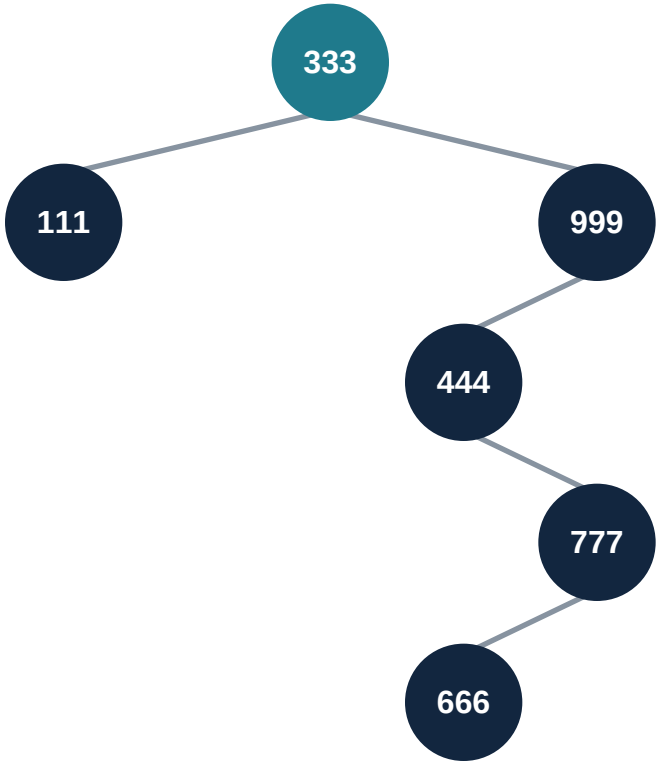
// Map: three views
for (Map.Entry<Integer, String> e : map.entrySet()) { e.getKey(); e.getValue(); }
for (Integer id   : map.keySet()) { ... }
for (String  name : map.values()) { ... }
```

Key points

- Every Collection is Iterable, so for-each works
- A Map is not Iterable: use entrySet(), keySet() or values()
- Do not remove with list.remove() inside for-each: use Iterator

TreeSet - Sorted Set (Tree Structure)

inserted: 333, 999, 444, 777, 111, 666 (simplified tree)



Key points

- Stores unique elements in SORTED order (natural or custom)
- Each add() compares the new element with existing ones to find its place
- Smaller goes left, bigger goes right; compare result 0 = duplicate, rejected
- Reading in order (in-order walk): 111, 333, 444, 666, 777, 999
- No null; elements must be Comparable or you supply a Comparator

Demo: TreeSetDemo - String, Integer, LocalDate

```
Set<String> hs = new TreeSet<String>();
hs.add("Ram");  hs.add("Sham");  hs.add("Krishna");
hs.add("Durga"); hs.add("Ganesh"); hs.add("Arjun");
for (String s : hs) { System.out.println(s); }

Set<Integer> nums = new TreeSet<Integer>();
nums.add(333); nums.add(999); nums.add(444);
nums.add(777); nums.add(111); nums.add(666);

Set<LocalDate> dates = new TreeSet<LocalDate>();
dates.add(LocalDate.now());
dates.add(LocalDate.of(2022, 4, 24));
dates.add(LocalDate.of(2021, 5, 12));
```

Key points

- Strings sort alphabetically, numbers ascending, dates oldest first
- These classes already implement Comparable

Output

```
Arjun
Durga
Ganesh
Krishna
Ram
Sham
[111, 333, 444, 666, 777, 999]
[2021-05-12, 2022-04-24, 2026-10-21]
(last value is today's date)
```

Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(Object o)	compare(Object a, Object b)
Defined	inside the class itself	in a separate class
Sort orders	ONE natural order	many custom orders
Used by	new TreeSet(), Collections.sort(list)	new TreeSet(cmp), Collections.sort(list, cmp)
Our demo	Employee (by eid)	EmployeeNameComparator (by name)

- Return value: negative = first is smaller, 0 = equal, positive = first is bigger

Comparable: Employee Natural Order

```
package c2;

public class Employee implements Comparable {
    private int eid;  private String name;  private double salary;
    // constructors, getters, setters, toString() ...

    @Override
    public int compareTo(Object o) {
        Employee e = (Employee) o;
        // natural sorting is generally done with eid
        int diff = this.getEid() - e.getEid();
        System.out.println("Current object eid " + this.getEid()
                + " copared with Second object " + e.getEid());
        if (diff > 0)      { return 1;  }
        else if (diff < 0) { return -1; }
        else               { return 0;  }
    }
}
```

Key points

- implements Comparable gives Employee a natural order
- this = current object, o = the other one
- TreeSet calls compareTo on every add()
- Shorter equivalent: return this.eid - e.eid;

Demo: EmployeeTreeSetDemo - Natural Sorting

```
Set<Employee> hs = new TreeSet<Employee>();

hs.add(new Employee(100, "Ram",     24000.00));
hs.add(new Employee(105, "Arjun",   22000.00));
hs.add(new Employee(109, "Sham",    28000.00));
hs.add(new Employee(104, "Krishna", 27000.00));
hs.add(new Employee(102, "Ganesh",  23000.00));

for (Employee e : hs) {
    System.out.println(e);
}
// sorted by eid, because Employee.compareTo() is used
```

Key points

- compareTo runs during add(), not during printing
- The very first add compares the element with itself (100 vs 100)
- Output is in eid order, not insertion order

Output

```
Current object eid 100 copared with Second object 100
Current object eid 105 copared with Second object 100
Current object eid 109 copared with Second object 100
... 6 more comparison lines ...
Employee [eid=100, name=Ram, salary=24000.0]
Employee [eid=102, name=Ganesh, salary=23000.0]
Employee [eid=104, name=Krishna, salary=27000.0]
Employee [eid=105, name=Arjun, salary=22000.0]
Employee [eid=109, name=Sham, salary=28000.0]
```

What if Employee is NOT Comparable?

```
// Employee WITHOUT  implements Comparable
Set<Employee> hs = new TreeSet<Employee>();
hs.add(new Employee(100, "Ram", 24000.00));    // throws here

Exception in thread "main" java.lang.ClassCastException:
    class Employee cannot be cast to class java.lang.Comparable
    at java.base/java.util.TreeMap.compare(TreeMap.java:1291)
    at java.base/java.util.TreeMap.put(TreeMap.java:536)
    at java.base/java.util.TreeSet.add(TreeSet.java:255)
    at cg1.EmployeeTreeSetDemo.main(EmployeeTreeSetDemo.java:16)
```

Key points

- TreeSet needs a way to order elements
- Without Comparable or a Comparator, add() fails at run time
- Two fixes: implement Comparable OR pass a Comparator to the TreeSet constructor
- Note: the stack trace shows TreeSet.add calling TreeMap.put - a TreeSet is built on a TreeMap

Comparator: Custom Sorting by Name

```
// separate class: custom sorting using Employee name
public class EmployeeNameComparator implements Comparator {
    @Override
    public int compare(Object o1, Object o2) {
        Employee e1 = (Employee) o1;
        Employee e2 = (Employee) o2;
        return e1.getName().compareTo(e2.getName());   // String's compareTo
    }
}

// EmployeeTreeSetDemo2
Comparator namecomp = new EmployeeNameComparator();
Set<Employee> hs = new TreeSet<Employee>(namecomp);   // pass to constructor
hs.add(new Employee(100, "Ram", 24000.00));           // ... same 5 employees
for (Employee e : hs) { System.out.println(e); }
```

Key points

- The Comparator is passed to the TreeSet constructor
- TreeSet now runs compare(), NOT Employee.compareTo()
- Two employees with the same name would count as duplicates: one is dropped

Output

```
Employee [eid=105, name=Arjun, salary=22000.0]
Employee [eid=102, name=Ganesh, salary=23000.0]
Employee [eid=104, name=Krishna, salary=27000.0]
Employee [eid=100, name=Ram, salary=24000.0]
Employee [eid=109, name=Sham, salary=28000.0]
```

Demo: EmployeeListSortDemo - Sorting a List

```
List<Employee> el = new ArrayList<Employee>();
el.add(new Employee(100, "Ram",     24000.00));
el.add(new Employee(105, "Arjun",   22000.00));
el.add(new Employee(109, "Sham",    28000.00));
el.add(new Employee(104, "Krishna", 27000.00));
el.add(new Employee(102, "Ganesh",  23000.00));
// Before sorting: insertion order

Collections.sort(el);          // natural sorting by eid (Comparable)

Comparator namecomp = new EmployeeNameComparator();
Collections.sort(el, namecomp);   // custom sorting by name (Comparator)
```

Key points

- A List keeps insertion order until you sort it
- Collections.sort(list) needs Comparable; sort(list, cmp) uses your Comparator
- TreeSet sorts as you add; a List sorts on demand

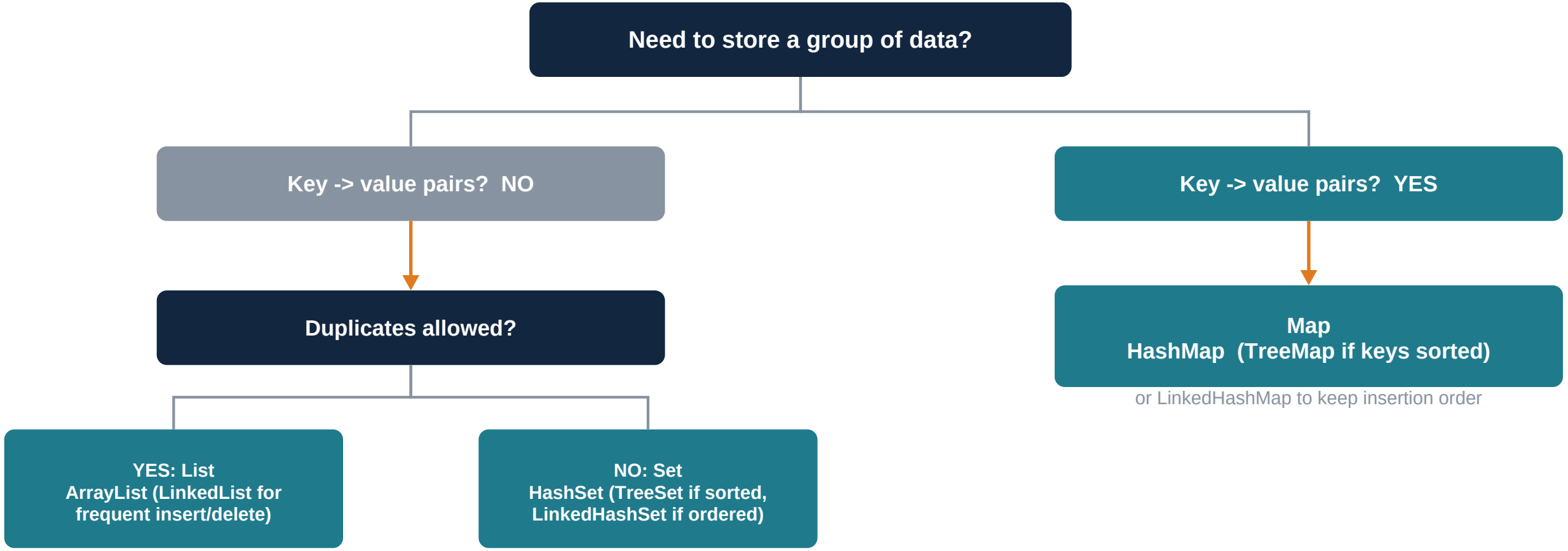
Output

```
Before : 100 Ram, 105 Arjun, 109 Sham,
         104 Krishna, 102 Ganesh
Natural: 100 Ram, 102 Ganesh, 104 Krishna,
         105 Arjun, 109 Sham
By name: 105 Arjun, 102 Ganesh, 104 Krishna,
         100 Ram, 109 Sham
(output condensed to ids and names)
```

Collections Utility Class (java.util.Collections)

Method	What it does
Collections.sort(list)	sorts using the natural order (Comparable)
Collections.sort(list, comparator)	sorts using your Comparator
Collections.reverse(list)	reverses the order
Collections.shuffle(list)	random order
Collections.max(c) / min(c)	largest / smallest element
Collections.unmodifiableList(list)	read-only view

Choosing the Right Collection



Assignment: EmployeeSalaryComparator

```
package c2;

public class EmployeeSalaryComparator implements Comparator {
    @Override
    public int compare(Object o1, Object o2) {
        // TODO 1: cast o1 and o2 to Employee
        // TODO 2: compare their salaries
        //         hint: Double.compare(e1.getSalary(), e2.getSalary())
        return 0;
    }
}

// TODO 3: use it in a TreeSet:      new TreeSet<Employee>(new EmployeeSalaryComparator())
// TODO 4: use it with a List:       Collections.sort(el, new EmployeeSalaryComparator())
```

Key points

- Sort employees by salary, lowest first
- Then make it highest first
- Test with both TreeSet and Collections.sort
- Challenge: if salaries are equal, sort by name

Best Practices

- Program to the interface: List<String> list = new ArrayList<>();
- Always use generics to get compile-time type safety
- ArrayList by default; HashSet/HashMap when order does not matter
- TreeSet/TreeMap only when you need sorted data
- Override equals() and hashCode() together for objects stored in hash-based collections
- Use Comparable for the natural order and Comparator for extra orders
- Use Iterator.remove() (not list.remove) while looping
- Choose the collection by how you use it: lookup, order, duplicates, sorting

Common Beginner Mistakes

- Using raw types (List list) and getting ClassCastException later
- Expecting a HashSet or HashMap to keep insertion order
- Calling get(i) on a Set (no index); use iteration or a List
- Using TreeSet with a class that is not Comparable: ClassCastException
- Comparator returning wrong sign, or treating equal sort keys as different objects
- Removing elements inside for-each: ConcurrentModificationException
- Confusing Collection (interface) with Collections (utility class)
- Putting null into a TreeSet / TreeMap key

Practice Questions

- 1. Draw the Collection hierarchy. Where does Map fit?
- 2. Difference between ArrayList and LinkedList? When to use each?
- 3. What happens when you add a duplicate to a List, a Set and a Map?
- 4. How do you update an element in a Set?
- 5. Why does HashSet output order differ from insertion order?
- 6. Difference between Comparable and Comparator? Give an example of each.
- 7. Why does TreeSet throw ClassCastException for a plain Employee?
- 8. Sort employees by salary descending using a Comparator.
- 9. Which collection would you use for: unique sorted names, ID -> name lookup, a to-do list?

Summary

- Collection framework = interfaces + implementations + algorithms in java.util
- List: ordered, duplicates, index | Set: unique | Map: key -> value
- ArrayList (fast read), LinkedList (fast insert), HashSet, TreeSet (sorted), HashMap, TreeMap
- Generics make collections type-safe
- TreeSet / TreeMap and Collections.sort need an ordering: Comparable or Comparator
- Collections class provides sort, reverse, max, min ...
- Pick the collection that matches how you will use the data